Project Part I — Language and Grammar Engineering

Grammar Engineering, 2026/27

Master's in Informatics Engineering, Universidade do Minho

- **Weight:** 50% of the practical/project assessment
- **Submission deadline:** November 4th
- **Project presentation:** November 5th
- **Submission:** Via Blackboard
- **Team size:** Group of 3 students

Objective

The goal of this project is to **design and specify a small computer language** and apply the concepts introduced in the first part of the course.

You will design a language for a specific domain or purpose, specify its lexical and syntactic structure, define relevant semantic properties using an attribute grammar, and evaluate the quality of the resulting language and grammar.

The project should demonstrate an integrated understanding of:

- language design;
- formal and context-free grammars;
- lexical and syntactic analysis;
- semantic elaboration;
- attribute grammars and attribute evaluation;
- language and grammar quality.

The language designed in this project will be reused in Part II of the project, where programs written in the language will be represented, analysed, visualized, and transformed.

Language domain and purpose

The language must be designed to address a specific domain or purpose.

The project should **not** consist simply of designing a small general-purpose imperative programming language. The language should have a clear intended purpose and its constructs should reflect the needs of its target domain.

The project should therefore identify:

- the intended domain and purpose of the language;
- the problems or tasks the language is intended to address;
- the intended users or use context, where relevant;
- the main concepts and abstractions represented by the language;
- how the language constructs support its intended purpose.

Appendix A provides examples of languages that may inspire possible project directions.

Minimum language requirements

The language must contain sufficient structure to support the concepts covered in Part I (and the subsequent analysis activities in Part II).

At minimum, it should include:

- **variables and declarations**;
- **scopes** and a clearly defined scoping discipline;
- **expressions** and some form of state or value manipulation;
- **compound and nested constructs**;
- meaningful **control-flow structure**, including some form of branching and repetition;
- semantic properties that cannot be determined from syntax alone.

The language should therefore be small enough to be specified and analysed within the scope of the project, while being sufficiently expressive to require non-trivial grammar and semantic specifications.

Tasks

1. **Language design**: Describe the purpose and intended domain of the language, and provide representative examples of valid programs.
2. **Lexical and syntactic specification**: Define the lexical structure of the language and specify the syntax using a context-free grammar.
3. **Semantic specification**: Identify semantic properties that cannot be determined from syntax alone, and specify selected properties using an attribute grammar.
4. **Language and grammar quality**: Evaluate the designed language and grammar according to relevant quality properties.

Note: The attribute grammar is not expected to cover all semantic properties of the language, but should be sufficient to demonstrate the use of attributes, semantic dependencies, and attribute evaluation.

Deliverables

The main project deliverable is a report discussing the outcomes of the tasks defined above and including the formal specification of the language, its context-free grammar, and its attribute grammar.

The project will be presented by the group in a 10-minute presentation, followed by a short discussion with the rest of the class.

Appendix A — Example Languages

The following languages are provided as inspiration for possible project directions. They illustrate different forms of domain-specific semantics and language design. Students are not expected to reproduce any of these languages.

Specialized programming languages

- **Dafny** — program verification
- **F*** — verified programming
- **Move** — resource-oriented programming
- **Triton** — GPU programming
- **P4** — programmable networking
- **Faust** — audio signal processing

Policy and configuration languages

- **Rego** — policy evaluation
- **Cedar** — authorization policies
- **Starlark** — build and configuration
- **Nix** — package and system configuration

Concurrent and workflow languages

- **Promela** — concurrent and distributed systems
- **WDL** — data-processing workflows
- **Nextflow** — computational workflows

Modeling and constraint languages

- **Alloy** — relational modeling
- **MiniZinc** — constraint programming
- **Statecharts** — state-based behavior

Query and transformation languages

- **SPARQL** — graph queries
- **jq** — JSON transformation
- **XSLT** — XML transformation